سند معماری

پروژهٔ کلاس‌یار

پلتفرم اشتراک جزوه، نمونه‌سؤال و ارزیابی استادان

تهیه‌کنندگان:
مریم قندی زنجانی، ملیکا شهبازی، آنیسا فروغمند، مائده رضایی، محمدمهدی ذوالقدر

دانشگاه آزاد زنجان

۱۱/۰۷/۱۴۰۵

فهرست مطالب

۱ مقدمه ۳
۱.۱ هدف سند . . . ۳
۲.۱ دامنه . . . ۳
۳.۱ ذی‌نفعان . . . ۳

۲ نمای کلی معماری ۴
۱.۲ سبک معماری . . . ۴
۲.۲ دیاگرام لایه‌ها . . . ۴
۳.۲ جریان داده . . . ۴

۳ لایه‌ها ۴
۱.۳ (Frontend) Layer Presentation . . . ۴
۲.۳ (Backend) Layer Business . . . ۵
۳.۳ (Database Layer Data و فایل) . . . ۵

۴ تصمیم‌های معماری (ADR) ۶
۱.۴ ADR-۰۰۱: چرا Monolith؟ . . . ۶
۲.۴ ADR-۰۰۲: چرا React؟ . . . ۶
۳.۴ ADR-۰۰۳: چرا PostgreSQL؟ . . . ۶
۴.۴ ADR-۰۰۴: ذخیرهٔ فایل جزوه‌ها . . . ۶

۵ API REST ۶
۱.۵ احراز هویت . . . ۶
۲.۵ جزوه‌ها . . . ۷
۳.۵ استادان و اطلاعات پایه . . . ۷
۴.۵ امتیاز، نظر و منابع . . . ۷
۵.۵ Endpointهای استاد . . . ۸
۶.۵ Endpointهای ادمین . . . ۸
۷.۵ نمونه Response . . . ۹

۶ امنیت ۱۰
۱.۶ احراز هویت . . . ۱۰
۲.۶ مجوزدهی . . . ۱۰
۳.۶ امنیت API . . . ۱۰
۴.۶ امنیت فایل و محتوا . . . ۱۰

۷ مقیاس‌پذیری ۱۰
۱.۷ محدودیت‌های فعلی ۱۰
۲.۷ مسیر رشد ۱۱

۸ ریسک‌ها و بدهی فنی ۱۱

۹ فناوری‌ها ۱۱

پیوست الف — تاریخچهٔ تغییرات ۱۲

نام پروژه	کلاس‌یار (ClassYar)
نسخهٔ سند	۱.۱
تاریخ	۰۲/۰۸/۱۴۰۵
تهیه‌کنندگان	مریم قندی زنجانی، ملیکا شهبازی، آنیسا فروغمند، مائده رضایی، محمدمهدی ذوالقدر
وضعیت	پیش‌نویس
مرجع	`docs/01-vision.md` (بخش ۸.۲) و `docs/02-requirements/user-stories.md`

---

۱ مقدمه

۱.۱ هدف سند

این سند معماری نرم‌افزار پروژهٔ **کلاس‌یار** را تعریف می‌کند. هدف آن ایجاد یک درک مشترک از ساختار سیستم، تصمیم‌های کلیدی و محدودیت‌ها است.

۲.۱ دامنه

- **در دامنه:** معماری سطح بالا، لایه‌ها، Endpointها، امنیت.
- **خارج از دامنه:** جزئیات پیاده‌سازی، کد.

۳.۱ ذی‌نفعان

ذی‌نفع	نقش
تیم توسعه (۵ نفر)	پیاده‌سازی
استاد درس	ارزیابی
کاربران هدف (دانشجو، استاد، مدیر)	استفاده و تأیید

---

۲ نمای کلی معماری

۱.۲ سبک معماری

Architecture Layered + Monolith

- یک برنامهٔ واحد
- سه لایهٔ اصلی: Presentation، Business، Data
- مناسب تیم کوچک (۵ نفر) و پروژهٔ ۱۴ هفته‌ای
- ساده برای شروع، قابل گسترش

۲.۲ دیاگرام لایه‌ها

```
+--------------+
|  Presentation Layer                    |
|  React (SPA)                           |
+-------+-------+
                      | HTTP / REST (JSON)
+-------v-------+
|  Business Layer                        |
|  Node.js + Express                     |
+-------+-------+
                      | Prisma (SQL)
+-------v-------+
|  Data Layer                            |
|  PostgreSQL + File Storage             |
+--------------+
```

۳.۲ جریان داده

کاربر ← مرورگر (React) ← API (Express) ← پایگاه داده (PostgreSQL) و ذخیرهٔ فایل

درخواست از کاربر به Frontend، سپس به API و در پایان به پایگاه داده می‌رسد؛ پاسخ همین مسیر را برمی‌گردد.

---

۳ لایه‌ها

۱.۳ Layer Presentation (Frontend)

مورد	انتخاب
فناوری	React ۱۸
Routing	Router React
State	Toolkit Redux
UI	CSS Tailwind (با پشتیبانی RTL)
Build	Vite

۲.۳ Layer Business (Backend)

مورد	انتخاب
فناوری	Node.js ۲۰ + Express ۴
ORM	Prisma
Validation	Zod
Auth	JWT
Logging	Winston
آپلود فایل	Multer

۳.۳ Layer Data (Database و فایل)

مورد	انتخاب
فناوری	PostgreSQL ۱۶
Migration	Migrate Prisma
ذخیرهٔ فایل	دیسک سرور، سپس سرویس ابری (MVP)
Backup	روزانه
Hosting	Railway

۴ تصمیم‌های معماری (ADR)

۱.۴ ADR-۰۰۱: چرا Monolith؟

تصمیم: معماری Monolith
دلیل: - تیم کوچک (۵ نفر) و زمان محدود (۱۴ هفته) - پیچیدگی کم و یک زبان (JavaScript) برای کل تیم - سرعت توسعه بالاتر - استقرار ساده

۲.۴ ADR-۰۰۲: چرا React؟

تصمیم: React برای Frontend
دلیل: - محبوب و مستند - اکوسیستم بزرگ - SPA برای تجربهٔ کاربری بهتر در جست‌وجو و فیلتر

۳.۴ ADR-۰۰۳: چرا PostgreSQL؟

تصمیم: PostgreSQL برای Database
دلیل: - رابطه‌های استاد، درس، واحد و جزوه رابطه‌ای هستند - قیدهای یکتایی (یک امتیاز برای هر کاربر) به‌سادگی اعمال می‌شود - رایگان و اوپن‌سورس

۴.۴ ADR-۰۰۴: ذخیرهٔ فایل جزوه‌ها

تصمیم: فایل‌ها روی دیسک سرور ذخیره می‌شوند و مسیر آن‌ها در PostgreSQL می‌ماند. دسترسی از طریق یک لایهٔ جداگانه (Storage Service) است.
دلیل: - ساده‌ترین راه برای MVP - با جداسازی لایه، گذر به سرویس ابری بدون تغییر در بقیهٔ کد ممکن است - محدودیت: فضای دیسک (حداکثر ۲۰ مگابایت برای هر فایل)

---

۵ API REST

همهٔ مسیرها با `/api` شروع می‌شوند. جست‌وجو و فیلتر با پارامترهای Query روی `GET /notes` انجام می‌شود (`professorId`، `provinceId`، `unitId`، `courseId`، `term`، `page`).

۱.۵ احراز هویت

Method	Endpoint	توضیح	Story
POST	/auth/register	ثبت‌نام	US-۰۱
POST	/auth/login	ورود	US-۰۱
POST	/auth/refresh	تمدید توکن	US-۰۱

۲.۵ جزوه‌ها

Method	Endpoint	توضیح	Story
GET	/notes	فهرست و فیلتر جزوه‌ها	US-۰۴، US-۰۷
GET	/notes/:id	جزئیات جزوه	US-۰۵
GET	/notes/:id/download	دانلود فایل	US-۰۵
POST	/notes	آپلود جزوه	US-۰۳
PUT	/notes/:id	ویرایش جزوهٔ خود	US-۰۶
DELETE	/notes/:id	حذف جزوهٔ خود	US-۰۶
GET	/me/notes	جزوه‌های من	US-۰۶

۳.۵ استادان و اطلاعات پایه

Method	Endpoint	توضیح	Story
GET	/professors	فهرست استادان	US-۱۰
GET	/professors/:id	صفحهٔ استاد	US-۱۰
POST	/professor-requests	درخواست افزودن استاد	US-۰۲
GET	/universities، /provinces، /courses /units،	فهرست اطلاعات پایه برای فیلترها	US-۰۷

۴.۵ امتیاز، نظر و منابع

Method	Endpoint	توضیح	Story
POST	/notes/:id/rating	ثبت یا ویرایش امتیاز جزوه	US-۰۸
POST	/professors/:id/rating	ثبت یا ویرایش امتیاز استاد	US-۱۱
GET	/notes/:id/comments	نظرهای جزوه	US-۰۹
POST	/notes/:id/comments	ثبت نظر برای جزوه	US-۰۹
GET	/professors/:id/comments	نظرهای استاد	US-۰۹
POST	/professors/:id/comments	ثبت نظر برای استاد	US-۰۹
POST	/comments/:id/replies	پاسخ به نظر	US-۰۹
GET	/professors/:id/resources	نمونه‌سؤال‌ها و لینک گروه‌ها	US-۱۲
POST	/professors/:id/resources	ثبت نمونه‌سؤال یا لینک گروه	US-۱۲
POST	/reports	گزارش جزوه یا نظر نامناسب	US-۱۴

۵.۵ Endpointهای استاد

Method	Endpoint	توضیح	Story
GET	/professor/notes	جزوه‌های مربوط به درس‌های استاد	US-۱۳
POST	/professor/notes/:id/approve	تأیید جزوه	US-۱۳
POST	/professor/notes/:id/bonus	ثبت نمرهٔ تشویقی	US-۱۳

۶.۵ Endpointهای ادمین

Method	Endpoint	توضیح	Story
GET	/admin/reports	فهرست گزارش‌ها	US-۱۵
PUT	/admin/reports/:id	بررسی یا رد گزارش	US-۱۵
DELETE	/admin/notes/:id	حذف جزوهٔ نامناسب	US-۱۵
DELETE	/admin/comments/:id	حذف نظر نامناسب	US-۱۵
GET	/admin/users	فهرست کاربران	US-۰۱، US-۱۵
PUT	/admin/users/:id	تأیید حساب استاد یا مسدودکردن	US-۰۱، US-۱۵
PUT / POST / DELETE	/admin/professors، /admin/provinces، /admin/universities، /admin/units، /admin/courses	مدیریت اطلاعات پایه	US-۰۲
PUT	/admin/professor-requests/:id	تأیید یا رد درخواست افزودن استاد	US-۰۲

۷.۵ نمونه Response

`GET /api/notes/1`

```
{
  "id": 1,
  "title": "Operations Research Notes",
  "professor": "Dr. Kazemi",
  "course": "Operations Research",
  "university": "Zanjan Azad University",
  "semester": 1,
  "rating": 4,
  "ratingCount": 12,
  "downloads": 120
}
```

---

۶ امنیت

۱.۶ احراز هویت

- **JWT** برای API
- **bcrypt** برای رمزنگاری گذرواژه (حداقل ۸ کاراکتر)
- **Token Refresh** برای تمدید

۲.۶ مجوزدهی

- **(RBAC) Control Access Role-Based**
- نقش‌ها: Admin Professor، Student، (و مهمان بدون توکن برای مشاهده)
- استاد فقط به جزوه‌های مربوط به درس خودش دسترسی دارد
- حساب استاد پس از تأیید مدیر فعال می‌شود

۳.۶ امنیت API

- **HTTPS** اجباری
- **Limiting Rate** (۱۰۰ درخواست در دقیقه)
- **CORS** محدود
- **Validation Input** با Zod

۴.۶ امنیت فایل و محتوا

- فقط PDF، JPG PNG و DOCX تا ۲۰ مگابایت؛ نوع فایل از محتوای آن بررسی می‌شود، نه فقط پسوند
- نام فایل پاک‌سازی می‌شود و فایل خارج از مسیر عمومی ذخیره می‌شود
- لینک گروه فقط با ساختار `https` ذخیره می‌شود
- نظرها قبل از نمایش برای جلوگیری از XSS پاک‌سازی می‌شوند

---

۷ مقیاس‌پذیری

۱.۷ محدودیت‌های فعلی

- Monolith ← مقیاس‌پذیری محدود

- یک سرور ← تک نقطهٔ شکست
- ذخیرهٔ فایل روی دیسک ← با رشد جزوه‌ها فضا کم می‌شود
- هدف فعلی: ۱۰۰ کاربر همزمان (NF-۰۲)

۲.۷ مسیر رشد

۱. فاز ۱ (MVP): Monolith + یک سرور + ذخیرهٔ فایل روی دیسک
۲. فاز ۲: انتقال فایل‌ها به سرویس ذخیرهٔ ابری
۳. فاز ۳: Load Balancer + چند سرور
۴. فاز ۴: Replication Database و جدا کردن سرویس‌های پرمصرف

---

۸ ریسک‌ها و بدهی فنی

ریسک	احتمال	اثر	راهکار
محتوای نامناسب یا ناقض حق مالکیت	بالا	بالا	قابلیت گزارش (F-۱۲) و پنل مدیر (F-۱۳)
پرشدن فضای ذخیره‌سازی	متوسط	بالا	سقف ۲۰ مگابایت و انتقال به سرویس ابری
امتیاز یا نظر جعلی	متوسط	متوسط	یک امتیاز برای هر کاربر، Limiting Rate
نفوذ امنیتی	کم	بالا	OWASP و اعتبارسنجی ورودی و فایل
از دست دادن داده	کم	بالا	Backup روزانه
بدهی فنی	بالا	متوسط	Refactor در Sprint ۳

---

۹ فناوری‌ها

لایه	فناوری
Frontend	React ۱۸ + Vite + Tailwind
Backend	Node.js ۲۰ + Express ۴
Database	PostgreSQL ۱۶
ORM	Prisma
Auth	bcrypt + JWT
ذخیرهٔ فایل	دیسک سرور (MVP)
Testing	Supertest + Jest
CI/CD	GitHub Actions
Deploy	Railway

پیوست الف — تاریخچهٔ تغییرات

نسخه	تاریخ	تغییرات	تهیه‌کننده
۱.۰	۲۹/۰۷/۱۴۰۵	نسخهٔ اولیه (مریم قندی زنجانی)	تیم کلاس‌یار
۱.۱	۰۲/۰۸/۱۴۰۵	اصلاح ساختار، Endpointها بر پایهٔ ۱۵ Story، ADR-۰۴، ریسک‌ها و امنیت فایل	تیم کلاس‌یار